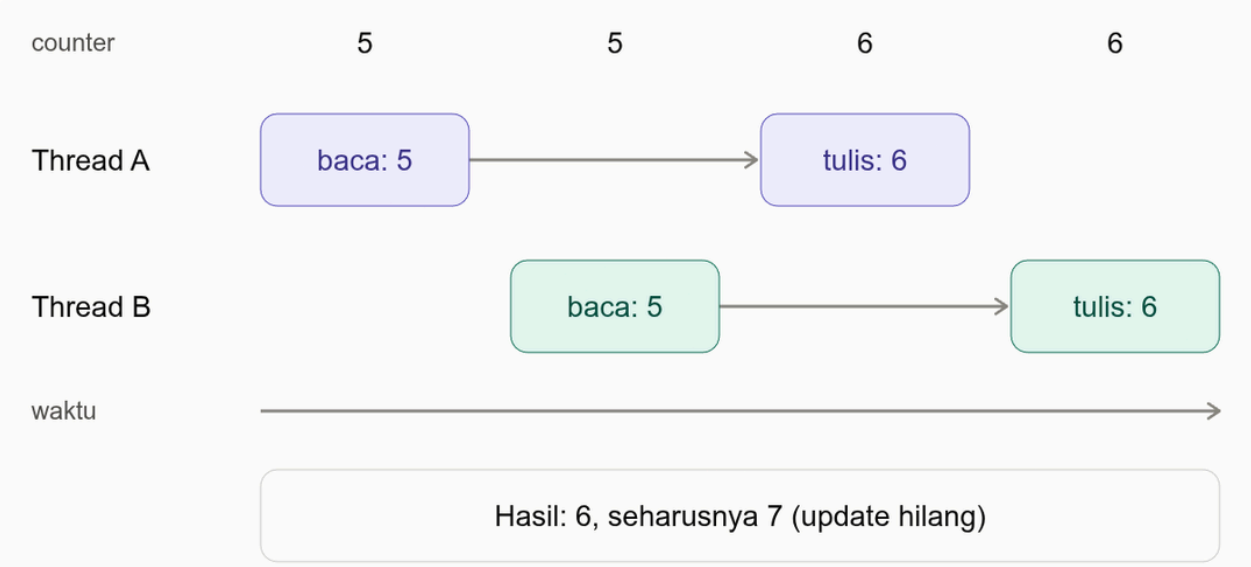
#6 Synchronization

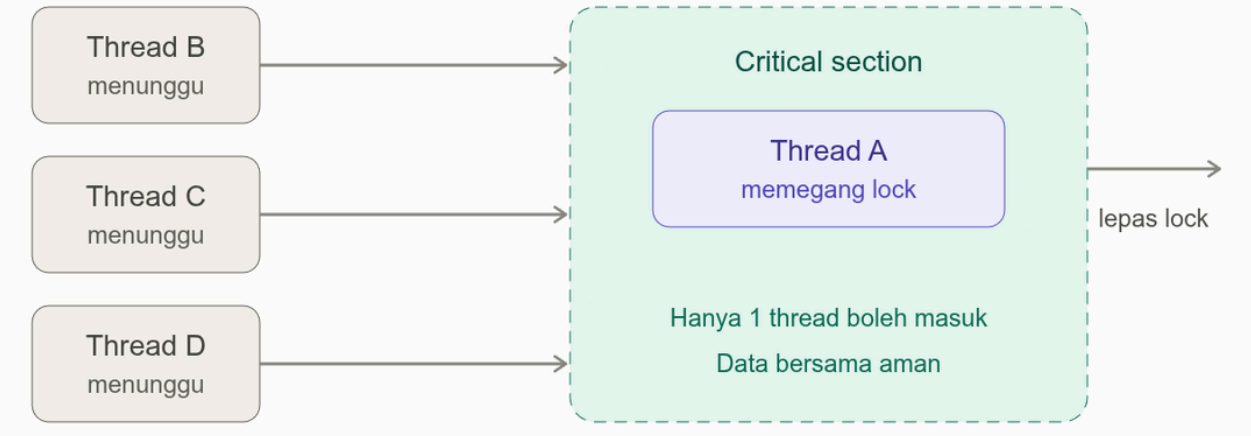
Race Condition

In parallel computing, threads or processes run simultaneously and often share data. When they read and write that data without coordination, the outcome depends on the scheduler's execution order.



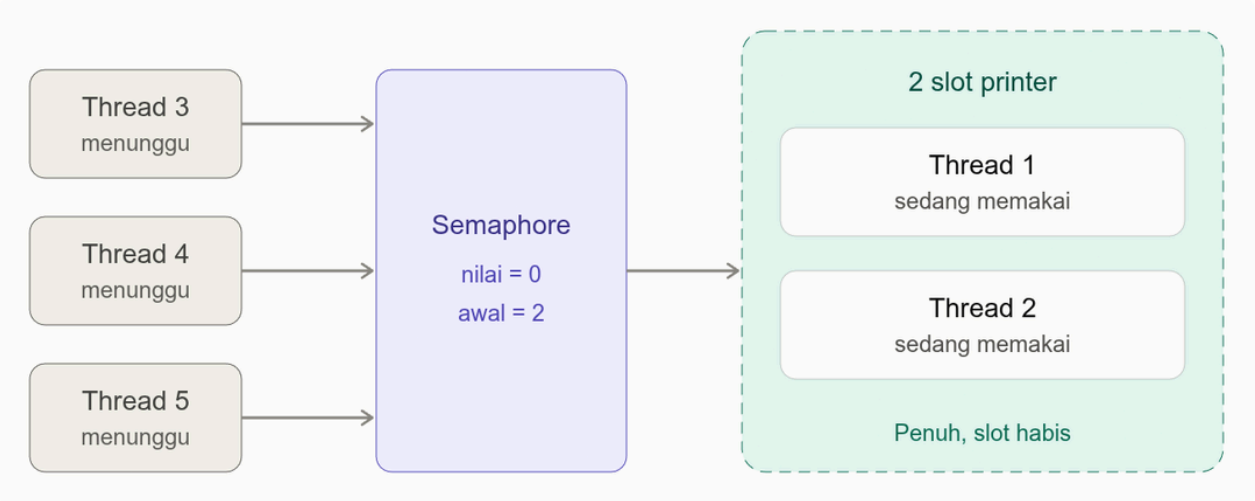
Mutual Exclusion

The principle that solves it: **only one thread may execute the critical section** (the code touching shared data) at a time. Others wait their turn. It is implemented with locks or mutexes. The cost is that protected code runs serially, so oversized or heavily contended critical sections hurt parallel performance.



Semaphore

A counter that controls how many threads may access a resource. wait decrements it (blocking at zero); signal increments it and wakes a waiter. Value 1 acts as a lock; value N limits access to N threads (e.g., 3 database connections). It also signals between threads, e.g., “consume only after the producer has produced.”



Labs

`race_condition.py`

Python

```python
import threading
import time

counter = 0
def tambah():
    global counter
    for _ in range(1000):
        temp = counter
        time.sleep(0)
        counter = temp + 1

t1 = threading.Thread(target=tambah)
t2 = threading.Thread(target=tambah)

t1.start()
t2.start()

t1.join()
t2.join()

print("Harapan :", 2000)
print("Hasil   :", counter, "<- salah karena race condition")
```

`mutex.py`

Python

```python
import threading
import time

counter = 0
mutex = threading.Lock()

def tambah():
    global counter
    for _ in range(1000):
        mutex.acquire()
        try:
            temp = counter
            time.sleep(0)
            counter = temp + 1
        finally:
            mutex.release()

t1 = threading.Thread(target=tambah)
t2 = threading.Thread(target=tambah)

t1.start()
t2.start()

t1.join()
t2.join()

print("Harapan :", 2000)
print("Hasil   :", counter, "<- Akurat karena Mutual Exclusion")
```

`semaphore.py`

Python

```python
import threading
import time

kuota_download = threading.Semaphore(2)

def download_file(nama_file):
    print(f"{nama_file} masuk antrean...")
    kuota_download.acquire()

    print(f"[MULAI] {nama_file} sedang didownload...")
    time.sleep(2)
    print(f"[SELESAI] {nama_file} berhasil diunduh.")

    kuota_download.release()

t1 = threading.Thread(target=download_file, args=("Thread
1: Film_A.mp4",))
t2 = threading.Thread(target=download_file, args=("Thread
2: Lagu_B.mp3",))
t3 = threading.Thread(target=download_file, args=("Thread
3: Tugas_C.pdf",))
t4 = threading.Thread(target=download_file, args=("Thread
4: Game_D.zip",))

t1.start()
t2.start()
t3.start()
t4.start()

t1.join()
t2.join()
t3.join()
t4.join()

print("\n Semua file selesai diproses!")
```